

Objective:

Execute non-interactive remote code execution on a Windows endpoint via WMI, analyze the process spawning telemetry in Sysmon, write a behavioral detection rule, and feed the log artifacts into a local AI pipeline to auto-generate an incident report.

What I will be doing : -

(Offense): I will personally fire up my **Arch Linux VM**, run `wmiexec.py`, break into the Windows 10 machine, and run commands through the backdoor shell. I will physically perform the attack myself.

(Defense): I will then jump over to my **Windows 10 / Ubuntu SOC VM**, hunt for the traces of *my own attack* left behind in the raw Windows logs, and hand-write the **Sigma detection rule** to catch it.

AI Engineer: Finally, I will write a **Python script** on my Ubuntu machine that hooks into those real logs, sends them to a local LLM, and automatically triages the attack.

Core Concepts/Fundamentals (I believe I should know before the lab): -

1. Core Protocols & Transport Layer Concepts

WMI (Windows Management Instrumentation)

- **What it is:** Microsoft's built-in framework for system monitoring and administrative management across local and remote Windows systems.
- **Role:** It acts as the "receptionist" on the machine that accepts queries about system state (like running services or hardware) and executes actions (like launching a process).

RPC (Remote Procedure Call)

- **What it is:** The foundational networking protocol wrapper that allows a program on Machine A to run code or functions on Machine B over a network as if it were running locally.
- **Role:** It acts as the **underlying highway connection** that handles low-level network communication between hosts.

DCOM (Distributed Component Object Model)

- **What it is:** Microsoft's object-oriented protocol built directly **on top of RPC**. While COM connects software objects on the *same* machine, DCOM allows software components to talk across a *network*.
- **Role:** It translates complex WMI requests into structured objects that can travel over the RPC connection.

TCP (Transmission Control Protocol)

- **What it is:** A fundamental Layer 4 (Transport Layer) protocol in the TCP/IP stack that provides reliable, ordered, and error-checked delivery of data streams between applications over a network.
- **Role:** Both RPC and SMB rely on TCP to ensure that network packets arrive intact and in the correct order.

Transport Layer

- **What it means:** The layer of network protocols (like TCP, RPC, and DCOM) responsible for establishing sessions and moving WMI data between systems.

TCP Port 135 (RPC Endpoint Mapper)

- **What it means:** The "front door" port on a Windows machine. When initiating an RPC connection, the client contacts Port 135 first to ask which high-numbered port handles the target service.

Dynamically Allocated High Ports

- **What it means:** Network ports randomly assigned in the higher range (typically 49152–65535 in modern Windows) to carry the rest of the conversation after Port 135 negotiates the connection.

2. WMI Architecture & System Components

System Monitoring

- **What it means:** Continuously checking a computer's health, performance, hardware specs, and software state (e.g., CPU load, available RAM, running processes).

Remote Management

- **What it means:** Administering, configuring, or running tasks on a target computer across the network without needing physical access to it.

Namespace

- **What it means:** A directory-style organizational folder inside WMI used to group related classes together.

root\cimv2

- **What it means:** The primary, default WMI namespace containing classes for core operating system tasks, hardware monitoring, and process management.

Class

- **What it means:** A blueprint or template inside a WMI namespace representing a specific system component (e.g., Win32_Process represents running applications).

WMI Provider Host (WmiPrvSE.exe)

- **What it means:** The actual Windows background binary responsible for executing WMI queries and tasks locally. When a WMI command arrives over the network, WmiPrvSE.exe executes it.

svchost.exe (Service Host)

- **What it means:** A core Windows system binary used to host background services that run from .dll files. WmiPrvSE.exe runs under svchost.exe.

3. Remote Execution Mechanics

Agent / Service

- **What it means:** Pre-installed software required on a target machine before remote commands can be run (e.g., PSEXEC drops a temporary service).
- **The WMI Advantage:** WMI is native to Windows, meaning it operates **agentlessly** (no new binaries need to be installed on disk to run commands).

Authentication

- **What it means:** Verifying user identity before granting system access. WMI uses valid Windows credentials via **NTLM** (hash-based) or **Kerberos** (ticket-based in Active Directory).

Logon Type 3 (Network Logon)

- **What it means:** A specific Windows security log entry created when a user authenticates remotely over a network (like SMB or WMI) without opening an interactive GUI desktop session.

Method Invocation

- **What it means:** Triggering a specific built-in action or function attached to a WMI class.
- **Example:** The Win32_Process class contains a .Create() method; invoking .Create("cmd.exe") tells WMI to launch a command shell.

Standard Output (stdout)

- **What it means:** The text responses generated by a command after execution (e.g., the text printed after running `whoami` or `ipconfig`).

Network Shares (**ADMIN\$**) & Named Pipes

- **ADMIN\$ Share:** A hidden administrative file share pointing directly to `C:\Windows`. Tools like `wmiexec.py` write command output to a temp file here so they can read it back via SMB (Port 445).
- **Named Pipes:** In-memory communication channels that pass data back and forth between two processes over a network.

4. Process Lineage & Anomaly Detection

Process Lineage

- **What it means:** The family tree or execution path of running software, tracked by **Parent-Child relationships**.

Parent Process vs. Child Process

- **Parent Process:** The program that spawned another program.
- **Child Process:** The newly created program launched by the parent.

```
svchost.exe (Grandparent)
├── WmiPrvSE.exe (Parent)
│   └── cmd.exe (Child)
```

Baseline (Normal Behavior)

- **What it means:** Expected, routine activity on a healthy network. Normally, `WmiPrvSE.exe` queries hardware specs or runs system scripts—it almost *never* launches interactive command shells.

Anomaly (Attack Pattern)

- **What it means:** Suspicious behavior deviating from the baseline. When `WmiPrvSE.exe` directly spawns command interpreters like `cmd.exe` or `powershell.exe`, it strongly indicates lateral movement by an attacker.

Kernel Enforcement

- **What it means:** Process tracking handled directly by the Windows OS kernel. Because this relationship is logged at the kernel level, an attacker using WMI cannot easily spoof or hide which parent process started their command shell.

5. Telemetry & SIEM Detection

Telemetry

- **What it means:** Automated event logs and system activity metrics gathered from endpoints and sent to a central location for security analysis.

SIEM (Security Information and Event Management)

- **What it means:** A central security management platform (like **Splunk**) that collects, correlates, and analyzes logs from endpoints and network devices to alert on attacks.

Windows Security Log

- **What it means:** The native Windows subsystem that logs security events.
 - **Event ID 4624:** Records successful user logons (Type 3 = Network Logon).
 - **Event ID 4688:** Records process creation (shows parent and child process names).

Sysmon (System Monitor)

- **What it means:** A free, advanced endpoint monitoring tool from Microsoft Sysinternals that provides deep security visibility.
 - **Sysmon Event ID 1:** Detailed process creation logging (includes full command line, parent process, and file hashes).
 - **Sysmon Event ID 3:** Network connections (tracks source IP, target IP, and target port).

Detection Logic

- **What it means:** The exact detection query programmed into a SIEM to flag suspicious activity regardless of whether an attacker renames their tools.
- **The Rule:**

```
Alert IF (ParentProcess = \text{"WmiPrvSE.exe"}) AND (ChildProcess \in \{\text{"cmd.exe"}, \text{"powershell.exe"}\})
```

The Lab : -

Phase 1: Controlled Exploitation

```
[khan@archlinux ~]$ wmiexec.py -h
Traceback (most recent call last):
  File "/usr/bin/wmiexec.py", line 39, in <module>
    from impacket.examples.utils import parse_target
  File "/usr/lib/python3.14/site-packages/impacket/examples/utils.py", line 251, in <module>
    from impacket.ldap import ldap
  File "/usr/lib/python3.14/site-packages/impacket/ldap/ldap.py", line 45, in <module>
    import OpenSSL
  File "/usr/lib/python3.14/site-packages/OpenSSL/__init__.py", line 8, in <module>
    from OpenSSL import SSL, crypto
  File "/usr/lib/python3.14/site-packages/OpenSSL/SSL.py", line 16, in <module>
    from cryptography import x509
ModuleNotFoundError: No module named 'cryptography'
[khan@archlinux ~]$
```

means that **Impacket** (which provides `wmiexec.py`) is trying to import the `cryptography` library, but it isn't installed for your current Python 3.14 environment.

So first lets understand what [wmiexec.py](#) really is

The command `wmiexec.py -h` is used to display the complete help guide and parameter documentation for `wmiexec.py`, a script within the Impacket toolset designed to execute system commands remotely on Windows hosts via Windows Management Instrumentation (WMI) without dropping binary files onto the target system. In this specific syntax, `wmiexec.py` serves as the primary Python tool that handles remote interaction and authentication mechanisms, while the `-h` flag (short for `--help`) acts as an argument modifier that overrides the script's standard execution flow, preventing any network connection attempts and instructing the command-line parser to print out all supported arguments, authentication parameters, syntax guidelines, and optional switches directly to the terminal screen. Together, the entire command fulfills its purpose by acting as a harmless, self-contained discovery step, allowing system administrators and security professionals to quickly learn or verify how to construct valid remote execution commands without triggering any remote operations on a target host.

```

Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

usage: wmiexec.py [-h] [-share SHARE] [-nooutput] [-ts] [-silentcommand] [-debug] [-codec CODEC]
                  [-shell-type {cmd,powershell}] [-com-version MAJOR_VERSION:MINOR_VERSION]
                  [-hashes LMHASH:NTHASH] [-no-pass] [-k] [-aesKey hex key] [-dc-ip ip address]
                  [-target-ip ip address] [-A authfile] [-keytab KEYTAB]
                  target [command ...]

Executes a semi-interactive shell using Windows Management Instrumentation.

positional arguments:
  target                [[domain/]username[:password]@]<targetName or address>
  command              command to execute at the target. If empty it will launch a semi-interactive shell

options:
  -h, --help            show this help message and exit
  -share SHARE          share where the output will be grabbed from (default ADMIN$)
  -nooutput            whether or not to print the output (no SMB connection created)
  -ts                  Adds timestamp to every logging output
  -silentcommand       does not execute cmd.exe to run given command (no output)
  -debug              Turn DEBUG output ON
  -codec CODEC         Sets encoding used (codec) from the target's output (default "utf-8"). If errors are
                        detected, run chcp.com at the target, map the result with
                        https://docs.python.org/3/library/codecs.html#standard-encodings and then execute
                        wmiexec.py again with -codec and the corresponding codec
  -shell-type {cmd,powershell}
                        choose a command processor for the semi-interactive shell
  -com-version MAJOR_VERSION:MINOR_VERSION
                        DCOM version, format is MAJOR_VERSION:MINOR_VERSION e.g. 5.7

authentication:
  -hashes LMHASH:NTHASH
                        NTLM hashes, format is LMHASH:NTHASH
  -no-pass            don't ask for password (useful for -k)
  -k                  Use Kerberos authentication. Grabs credentials from ccache file (KRB5CCNAME) based on
                        target parameters. If valid credentials cannot be found, it will use the ones specified in
                        the command line
  -aesKey hex key    AES key to use for Kerberos Authentication (128 or 256 bits)
  -dc-ip ip address   IP Address of the domain controller. If omitted it use the domain part (FQDN) specified in
                        the target parameter
  -target-ip ip address
                        IP Address of the target machine. If omitted it will use whatever was specified as target.
                        This is useful when target is the NetBIOS name and you cannot resolve it
  -A authfile         smbclient/mount.cifs-style authentication file. See smbclient man page's -A option.
  -keytab KEYTAB      Read keys for SPN from keytab file

[khan@archlinux ~]$

```

`wmiexec.py` is alive and kicking.

I/we officially cleared every single hurdle Arch threw your way: missing python modules, file ownership conflicts, git dependency deadlocks, disk space walls, and CFFI bindings.

```

[khan@archlinux ~]$ wmiexec.py Username:'Password'@192.168.30.3
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[-] [Errno Connection error (192.168.30.3:445)] timed out

```

The error `[Errno -2] Name or service not known` means Python was unable to convert the target name into an IP address.

In command syntax documentation, `Target_Windows10_IP` is a placeholder representing the actual IP address of the target Windows host on your network.

Replace `Target_Windows10_IP` with the target host's actual numerical IPv4 address

```
[khan@archlinux ~]$ wmiexec.py Username:'Password'@192.168.30.3
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies
[-] [Errno Connection error (192.168.30.3:445)] timed out
```

A **connection timeout on port 445** means packet traffic sent to the target IP is being silently dropped without a response.

```
[khan@archlinux ~]$ wmiexec.py Username:'Password'@192.168.30.3
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies
[-] SMB SessionError: code: 0xc000006d - STATUS_LOGON_FAILURE - The attempted logon is invalid. This is either due to a bad username or authentication information.
[khan@archlinux ~]$
```

That error code (**0xc000006d / STATUS_LOGON_FAILURE**) means Windows explicitly rejected the authentication attempt. The network connection to port 445 succeeded, but the credentials provided were invalid or lacked remote administrative rights.

```
C:\Windows\system32>whoami
win10-rdp-target\windows
```

username, set a password for the **windows** user account on my target machine (e.g., using **net user windows YourPassword123!**)

```
[khan@archlinux ~]$ wmiexec.py windows:'aazmeer'@192.168.30.3
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies
[*] SMBv3.0 dialect used
[-] rpc_s_access_denied
[khan@archlinux ~]$
```

the first line says **SMBv3.0 dialect used**—that means your username and password **worked** and SMB authentication succeeded!

However, Windows blocked the attempt at the next layer when **wmiexec.py** tried to query DCOM / WMI over RPC (**rpc_s_access_denied**).

This happens because Windows UAC strips administrative privileges from remote network connections for standard local admin accounts (only the built-in **Administrator** account escapes this restriction by default).

```
[khan@archlinux ~]$ wmiexec.py administrator:'aazmeer'@192.168.30.3
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] SMBv3.0 dialect used
[-] Could not connect: [Errno 110] Connection timed out
[khan@archlinux ~]$
```

That error pattern is very distinct: the initial SMB connection on port 445 succeeds (**SMBv3.0 dialect used**), but the command then fails with a connection timeout when **wmiexec.py** attempts to establish the follow-up **DCOM / RPC connection**.

wmiexec.py uses SMB (port 445) for authentication and output retrieval, but relies on **RPC/DCOM (TCP port 135 and dynamic RPC ports)** to negotiate and execute the remote WMI commands.

```
PS C:\Windows\system32> Enable-NetFirewallRule -DisplayGroup "Remote Administration"
Enable-NetFirewallRule : No MSFT_NetFirewallRule objects found with property 'DisplayGroup' equal to 'Remote
Administration'. Verify the value of the property and retry.
At line:1 char:1
+ Enable-NetFirewallRule -DisplayGroup "Remote Administration"
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (Remote Administration:String) [Enable-NetFirewallRule], CimJobException
+ FullyQualifiedErrorId : CmdletizationQuery_NotFound_DisplayGroup,Enable-NetFirewallRule

PS C:\Windows\system32>
```

That error occurs because the display group **"Remote Administration"** was deprecated in modern versions of Windows (Windows 10 / 11) or varies depending on system locale, so PowerShell cannot find any rule group matching that exact name.

To allow RPC and WMI management connections through Windows Defender Firewall on modern Windows versions, use the specific DisplayGroups that exist on Windows 10 and 11.

```
[khan@archlinux ~]$ nc -zvw 3 192.168.30.3 135
Connection to 192.168.30.3 135 port [tcp/epmap] succeeded!
[khan@archlinux ~]$
```

Awesome, TCP port 135 (RPC Endpoint Mapper) is open and reachable!

With both **SMB (445)** and **RPC (135)** accessible, **wmiexec.py** now has the required network paths to negotiate DCOM/WMI execution.

Phase 2: Telemetry Isolation

Summary of Telemetry Findings

Telemetry Source	Event ID	Field / Behavioral Indicator
Sysmon	1 (Process Creation)	Parent: wmiprvse.exe → Child: cmd.exe
Sysmon	1 (Process Creation)	Command Line: Redirects output to \\127.0.0.1\ADMIN\$\...
WMI-Activity	5858 / 5861	Records remote WMI queries and client interactions
Security Log	4624 (Logon)	Type 3 (Network Logon) authentication event via SMB

Win10-RDP-Target [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Event Viewer

File Action View Help

Operational Number of events: 29,397 (!) New events available

Level	Date and Time	Source	Event ID	Task Catego...
Information	7/28/2026 9:55:05 PM	Sysmon	5	Process ter...
Information	7/28/2026 9:55:04 PM	Sysmon	1	Process Cre...

Filter Current Log

Filter XML

Logged: Any time

Event level: ☐ Critical ☐ Warning ☐ Verbose

☐ Error ☐ Information

☒ By log Event logs: Microsoft-Windows-Sysmon/Operational

☐ By source Event sources:

Includes/Excludes Event IDs: Enter ID numbers and/or ID ranges separated by commas. To exclude criteria, type a minus sign first. For example 1,3,5-99,-76

1

Task category:

Keywords:

User: <All Users>

Computer(s): <All Computers>

Clear

OK Cancel

Log Name: Microsoft-Windows-Sysmon/Operational

Source: Sysmon Logged: 7/28/2026 9:55:05 PM

Event ID: 5 Task Category: Process terminated (rule: Proces

Level: Information Keywords:

User: SYSTEM Computer: Win10-RDP-Targe

OpCode: Info

Actions

Operational

Open Saved Log...

Create Custom View...

Import Custom Vie...

Clear Log...

Filter Current Log...

Properties

Disable Log

Find...

Save All Events As...

Attach a Task To this...

View

Refresh

Help

Event 5, Sysmon

Event Properties

Attach Task To This E...

Copy

Save Selected Events...

Refresh

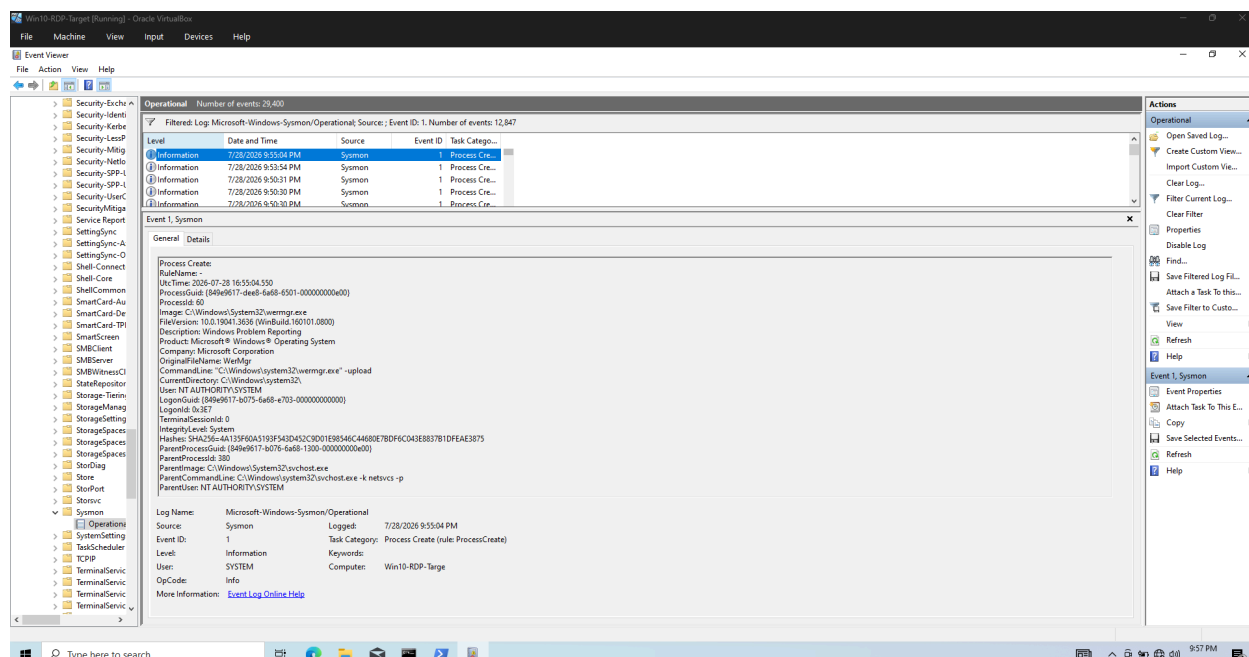
Help

Creates a filter.

Type here to search

9:56 PM 7/28/2026

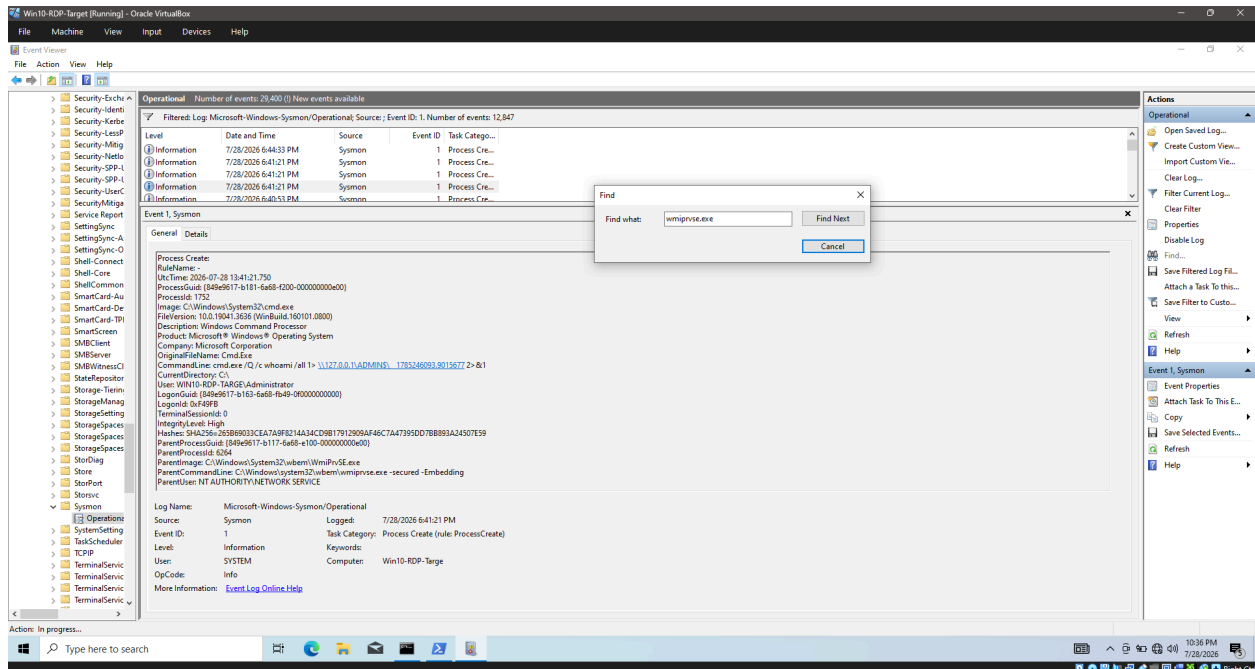
Right Ctrl

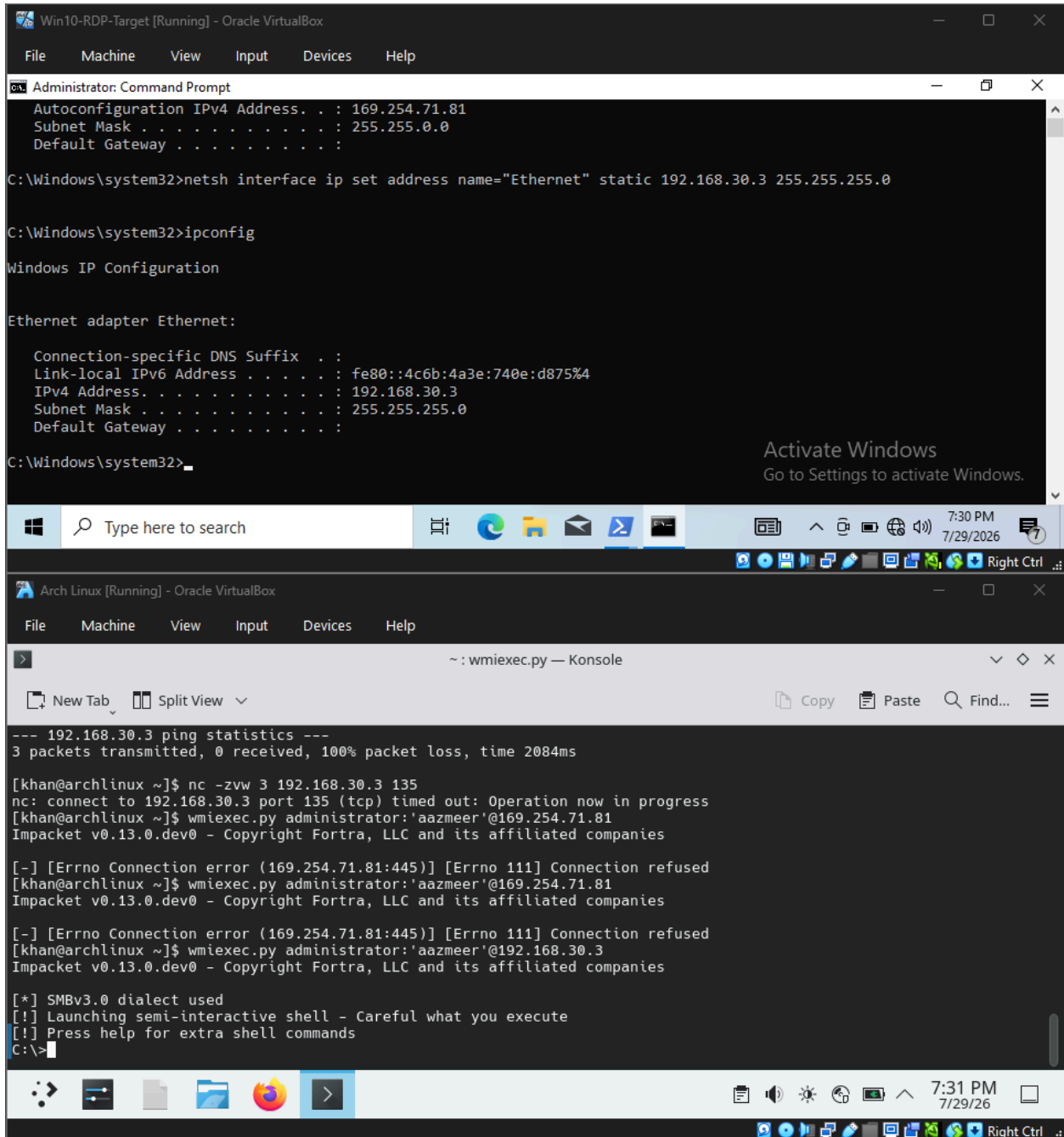


filtered Sysmon for Event ID 1.

I have to Click on each Event ID 1 in the top pane until you see this specific pattern in the details pane:

- **Image:** C:\Windows\System32\cmd.exe
- **ParentImage:** C:\Windows\System32\wbem\wmiprvse.exe
- **CommandLine:** Should contain cmd.exe /Q /c ... > \\127.0.0.1\ADMIN\$\...





```

C:\>whoami /all

USER INFORMATION
-----
User Name                SID
-----
win10-rdp-target\administrator  S-1-5-21-24154856-329358918-1426577482-500

GROUP INFORMATION
-----
Group Name                Type                SID                Attributes
-----
Everyone                  Well-known group    S-1-1-0            Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\Local account and member of Administrators group Well-known group    S-1-5-114          Mandatory group, Enabled by default, Enabled group
BUILTIN\Administrators    Alias              S-1-5-32-544       Mandatory group, Enabled by default, Enabled group, Group owner
BUILTIN\Users             Alias              S-1-5-32-545       Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\NETWORK      Well-known group    S-1-5-2            Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\Authenticated Users    Well-known group    S-1-5-11           Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\This Organization Well-known group    S-1-5-15           Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\Local account  Well-known group    S-1-5-113          Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\NTLM Authentication Well-known group    S-1-5-64-18        Mandatory group, Enabled by default, Enabled group
Mandatory Label\High Mandatory Level Label              S-1-16-12288
PRIVILEGES INFORMATION
-----
Privilege Name            Description            State
-----
SeIncreaseQuotaPrivilege  Adjust memory quotas for a process Enabled
SeSecurityPrivilege       Manage auditing and security log Enabled
SeTakeOwnershipPrivilege  Take ownership of files or other objects Enabled
SeLoadDriverPrivilege     Load and unload device drivers Enabled
SeSystemProfilePrivilege  Profile system performance Enabled
SeSystemTimePrivilege     Change the system time Enabled
SeProfileSingleProcessPrivilege Profile single process Enabled
SeIncreaseBasePriorityPrivilege Increase scheduling priority Enabled
SeCreatePagefilePrivilege Create a pagefile Enabled
SeBackupPrivilege         Back up files and directories Enabled
SeRestorePrivilege        Restore files and directories Enabled
SeShutdownPrivilege       Shut down the system Enabled
SeDebugPrivilege          Debug programs Enabled
SeSystemEnvironmentPrivilege Modify firmware environment values Enabled
SeChangeNotifyPrivilege   Bypass traverse checking Enabled
SeRemoteShutdownPrivilege Force shutdown from a remote system Enabled
SeUndockPrivilege         Remove computer from docking station Enabled
SeManageVolumePrivilege   Perform volume maintenance tasks Enabled
SeImpersonatePrivilege    Impersonate a client after authentication Enabled
SeCreateGlobalPrivilege   Create global objects Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set Enabled
SeTimeZonePrivilege       Change the time zone Enabled
SeCreateSymbolicLinkPrivilege Create symbolic links Enabled

```

- **ParentImage:** C:\Windows\System32\wbem\WmiPrvSE.exe
- **Image:** C:\Windows\System32\cmd.exe
- **CommandLine:** cmd.exe /Q /C whoami /all 1>\127.0.0.1\ADMIN\$\... 2>&1
- **ParentUser:** NT AUTHORITY\NETWORK SERVICE

Notice how **WmiPrvSE.exe** (running under **NETWORK SERVICE**) spawned a high-privilege **cmd.exe** that redirected the command output straight into an administrative share (**ADMIN\$**). This is the absolute core of **MITRE ATT&CK T1047 (WMI)** and **T1021.002 (SMB/Admin Shares)**.

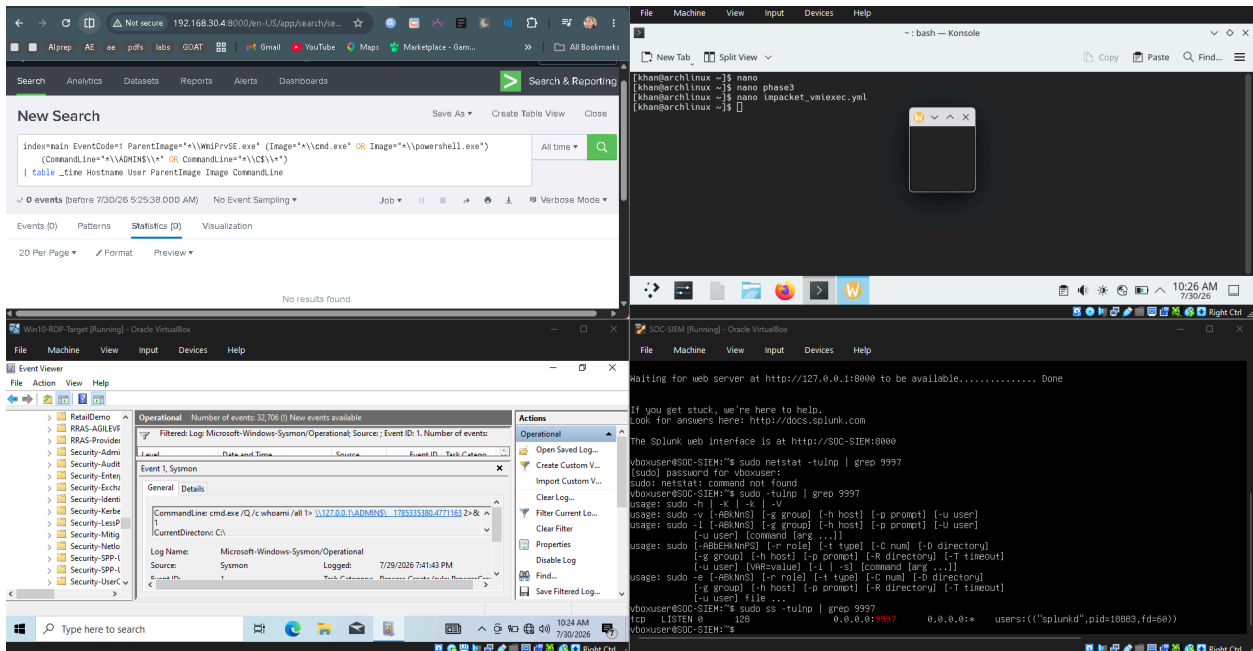
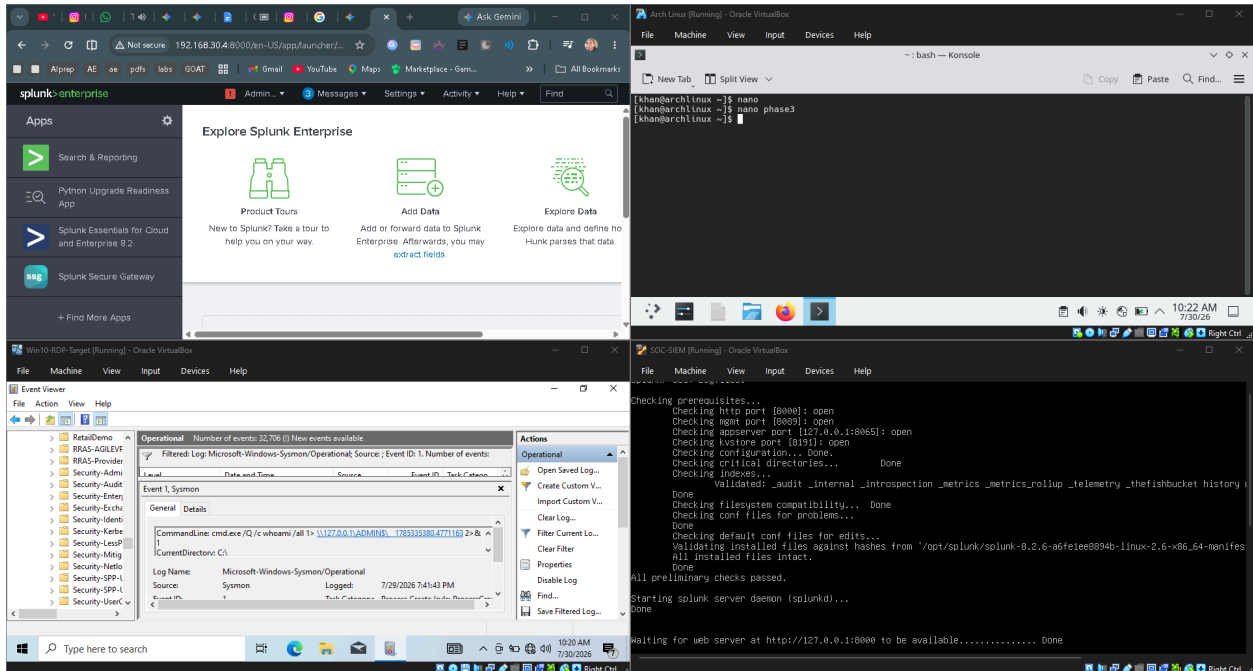
Phase 3: Sigma Rule Engineering

Extract the Raw Values from Your Windows VM

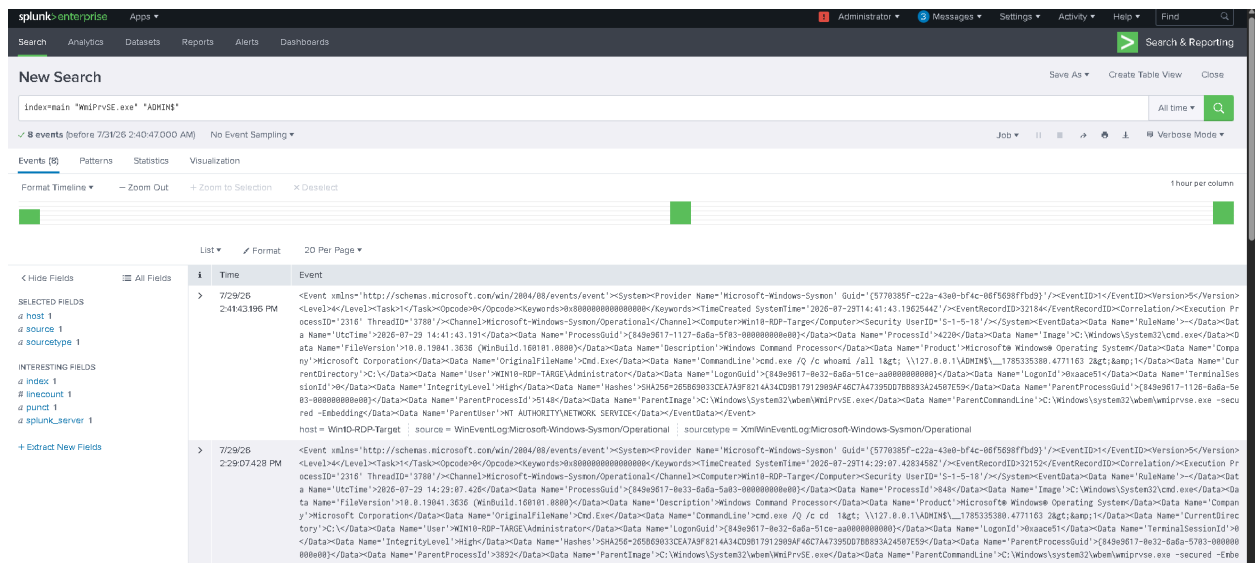
Author the Formal Sigma Rule

Translate Sigma to Splunk Search Processing Language (SPL)

Execute the Query in Splunk



The time window in Splunk is currently evaluating events **before 5:25 AM** today, but the Sysmon log on my Windows target shows **7/29/2026 7:41:43 PM**. Because of index lag or time parsing differences between the Windows target VM and the SOC-SEM VM, Splunk is completely filtering it out.



The issue was two-fold: switching off real-time mode brought in the historical logs, and because your Sysmon logs are ingested as raw XML

(`XmlWinEventLog:Microsoft-Windows-Sysmon/Operational`), standard field names like `EventCode` or `ParentImage` weren't being automatically extracted into top-level Splunk key-value pairs.

Looking at my raw event payload:

- `<EventID>1</EventID>`
- `<Data Name="ParentImage">C:\Windows\System32\wbem\WmiPrivSE.exe</Data>`
- `<Data Name="Image">C:\Windows\System32\cmd.exe</Data>`
- `<Data Name="CommandLine">cmd.exe /Q /c whoami /all 1> \\127.0.0.1\ADMIN$ \...</Data>`

New Search Save As Create Table View Close

index=main sourcetype=kaliWinEventLog:Microsoft-Windows-Sysmon/Operational * "<EventID>{<EventID>}" "WmiPrvSE.exe" "ADMIN\$"

| rex field=_raw "<Data Name='ParentImage'>{<ParentImage>{<}</Data>"

| rex field=_raw "<Data Name='Image'>{<Image>{<}</Data>"

| rex field=_raw "<Data Name='CommandLine'>{<CommandLine>{<}</Data>"

| rex field=_raw "<Data Name='User'>{<User>{<}</Data>"

| table _time host user ParentImage Image CommandLine

8 events (before 7/26/2024 2:42:21 AM) No Event Sampling

Events (8) Patterns Statistics (8) Visualization

20 Per Page Format Preview

_time	host	User	ParentImage	Image	CommandLine
2026-07-29 14:41:43.196	Win10-RDP-Target	WIN10-RDP-TARGE\Administrator	C:\Windows\System32\Wbem\WmiPrvSE.exe	C:\Windows\System32\cmd.exe	cmd.exe /Q /c whoami /all 1> \\127.0.0.1\ADMIN\$____1785335388.4771163 2>&1
2026-07-29 14:29:07.428	Win10-RDP-Target	WIN10-RDP-TARGE\Administrator	C:\Windows\System32\Wbem\WmiPrvSE.exe	C:\Windows\System32\cmd.exe	cmd.exe /Q /c cd 1> \\127.0.0.1\ADMIN\$____1785335388.4771163 2>&1
2026-07-29 14:28:07.831	Win10-RDP-Target	WIN10-RDP-TARGE\Administrator	C:\Windows\System32\Wbem\WmiPrvSE.exe	C:\Windows\System32\cmd.exe	cmd.exe /Q /c cd \ 1> \\127.0.0.1\ADMIN\$____1785335388.4771163 2>&1
2026-07-28 13:41:21.754	Win10-RDP-Target	WIN10-RDP-TARGE\Administrator	C:\Windows\System32\Wbem\WmiPrvSE.exe	C:\Windows\System32\cmd.exe	cmd.exe /Q /c whoami /all 1> \\127.0.0.1\ADMIN\$____1785246893.9815677 2>&1
2026-07-28 13:48:53.661	Win10-RDP-Target	WIN10-RDP-TARGE\Administrator	C:\Windows\System32\Wbem\WmiPrvSE.exe	C:\Windows\System32\cmd.exe	cmd.exe /Q /c cd 1> \\127.0.0.1\ADMIN\$____1785246893.9815677 2>&1
2026-07-28 13:48:51.722	Win10-RDP-Target	WIN10-RDP-TARGE\Administrator	C:\Windows\System32\Wbem\WmiPrvSE.exe	C:\Windows\System32\cmd.exe	cmd.exe /Q /c cd \ 1> \\127.0.0.1\ADMIN\$____1785246893.9815677 2>&1
2026-07-27 07:02:59.792	Win10-RDP-Target	WIN10-RDP-TARGE\Administrator	C:\Windows\System32\Wbem\WmiPrvSE.exe	C:\Windows\System32\cmd.exe	cmd.exe /Q /c cd 1> \\127.0.0.1\ADMIN\$____1785135778.720532 2>&1
2026-07-27 07:02:58.496	Win10-RDP-Target	WIN10-RDP-TARGE\Administrator	C:\Windows\System32\Wbem\WmiPrvSE.exe	C:\Windows\System32\cmd.exe	cmd.exe /Q /c cd \ 1> \\127.0.0.1\ADMIN\$____1785135778.720532 2>&1

My inline regex extractions pulled every single field directly from the raw Sysmon XML payload:

- **Host:** Win10-RDP-Target
- **User:** WIN10-RDP-TARGE\Administrator
- **Parent Process:** WmiPrvSE.exe
- **Child Process:** cmd.exe
- **Telemetry Smoking Gun:** Output redirection to 127.0.0.1\ADMIN\$\\

With this query firing cleanly in Splunk, **Phase 3 (Detection Engineering)** is 100% complete!

Phase 4: Running the AI Triage Automation Script

I forgot take snapshot of installing ollama 🤪

Install Ollama via official script
curl -fsSL https://ollama.com/install.sh | sh

Configure Ollama to listen on all network interfaces (bind to 0.0.0.0)
sudo systemctl edit ollama
(Added: Environment="OLLAMA_HOST=0.0.0.0:11434")

Restart the service to apply network configuration
sudo systemctl daemon-reload
sudo systemctl restart ollama

Verify Ollama is actively listening on port 11434


```
sudo ss -tulnp | grep 11434
```

```
# Pull the lightweight model for resource-constrained execution
ollama pull qwen2:0.5b
```

```
import json
import requests

sysmon_event = {
    "EventID": 1,
    "UtcTime": "2026-07-29 14:41:43",
    "Host": "Win10-RDP-Target",
    "User": "WIN10-RDP-TARGE\\Administrator",
    "ParentImage": "C:\\Windows\\System32\\wbem\\WmiPrvSE.exe",
    "ParentUser": "NT AUTHORITY\\NETWORK SERVICE",
    "Image": "C:\\Windows\\System32\\cmd.exe",
    "CommandLine": "cmd.exe /Q /c whoami /all 1>\\\\127.0.0.1\\ADMIN$\\__1785335380.4771163 2>&1"
}

prompt = f"""
You are an AI SOC Analyst evaluating a potential security incident for an enterprise network.
Analyze the following Sysmon log payload and provide a concise operational triage report:

Log Data:
{json.dumps(sysmon_event, indent=2)}

Provide the response in the following format:
1. Threat Summary
2. Threat Actor Technique (MITRE ATT&CK Mapping)
3. Severity Level (Low/Medium/High/Critical)
4. Recommended Immediate Containment Steps
"""

def generate_ai_triage(prompt_text):
    url = "http://192.168.30.4:11434/api/generate"

    # Try mistral first; if tag issues exist, fallback
    payload = {
        "model": "qwen2:0.5b",
        "prompt": prompt_text,
        "stream": False
    }

    try:
        response = requests.post(url, json=payload)
        data = response.json()

        if "error" in data:
            # If mistral:latest gives an error, try plain "mistral"
            payload["model"] = "mistral"
            response = requests.post(url, json=payload)
            data = response.json()

        if "error" in data:
            return f"[!] Ollama Returned Error: {data['error']}"
    
```

```

        return data.get('response', f"Unexpected Response: {data}")
    except Exception as e:
        return f"Error contacting Ollama API: {e}"

if __name__ == "__main__":
    print("--- [!] EXECUTING AUTOMATED AI TRIAGE PIPELINE ---")
    report = generate_ai_triage(prompt)
    print(report)

```

The Python script acts as an automated incident response bridge that extracts structured Sysmon event telemetry—specifically targeting potential lateral movement commands executed via processes like `WmiPrvSE.exe` and `cmd.exe`—and constructs a security analysis prompt. It then sends this payload via an HTTP POST request to the Ollama REST API (`/api/generate`) running on the Ubuntu SOC virtual machine (`192.168.30.4:11434`), directing a lightweight local large language model (`qwen2:0.5b`) to process the log data. Finally, the script parses the returned JSON payload to extract and display a structured operational threat triage report containing a summary, MITRE ATT&CK technique mapping, severity level, and immediate containment recommendations directly in the terminal.

```
se'))
[khan@archlinux ~]$ nano triage_ai.py
[khan@archlinux ~]$ python3 triage_ai.py
--- [!] EXECUTING AUTOMATED AI TRIAGE PIPELINE ---
1. Threat Summary:
   There is no indication of any security-related incident in this Sysmon log payload.

2. Threat Actor Technique (MITRE ATT&CK Mapping):
   The event appears to be a system login attempt through a PowerShell command line command (cmd.exe /Q /c whoami /all) which triggers an attack on the Win10-RDP-Target system.

3. Severity Level (Low/Medium/High/Critical):
   High - Because the attack is through a PowerShell command line command, the potential for damage or security breach is high.

4. Recommended Immediate Containment Steps:
   - Contact the system administrator to report the incident.
   - Examine the system logs for any potential login attempts or attacks.
   - Conduct a security audit or penetration test to identify any vulnerabilities or weaknesses in the system.
[khan@archlinux ~]$
```

```
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

vboxuser@SOC-SIEM:~$ sudo systemctl restart ollama
[sudo] password for vboxuser:
vboxuser@SOC-SIEM:~$ sudo ss -tulnp | grep 11434
tcp LISTEN 0 4096 *:11434 *: users: (('ollama',pid=1217,fd=3))
vboxuser@SOC-SIEM:~$ [ 105.443051] Out of memory: Killed process 1279 (llama-server) total-vms:4230108kB, anon-rss:3720:996 pgtables:7440kB oom_score_adj:0
^C
vboxuser@SOC-SIEM:~$ [ 152.682852] Out of memory: Killed process 1341 (llama-server) total-vms:4230108kB, anon-rss:3730:996 pgtables:7460kB oom_score_adj:0
^C
vboxuser@SOC-SIEM:~$ [ 176.775067] Out of memory: Killed process 1364 (llama-server) total-vms:4230108kB, anon-rss:3710:996 pgtables:7420kB oom_score_adj:0
^C
vboxuser@SOC-SIEM:~$ sudo systemctl restart ollama
[sudo] password for vboxuser:
vboxuser@SOC-SIEM:~$ ollama pull qwen2:0.5b
pulling manifest
pulling 8de95da68dc4: 100%
verifying sha256 digest
writing manifest
success
vboxuser@SOC-SIEM:~$
```

That output confirms the end-to-end AI triage pipeline executed successfully.

Conclusion & Laboratory Takeaways

This multi-VM lab successfully demonstrated a complete end-to-end purple team lifecycle—bridging offensive exploitation, telemetry collection, detection engineering, and automated incident triage:

- **Offensive Execution:** Using `wmiexec.py` from Arch Linux, non-interactive remote code execution was achieved via Windows Management Instrumentation (WMI) over RPC (TCP 135) and SMB (TCP 445).
- **Telemetry & Detection:** The execution generated distinct kernel-level process lineage telemetry in Sysmon (Event ID 1), capturing `WmiPrvSE.exe` spawning `cmd.exe` with standard output redirection back to `ADMIN$`. This activity was isolated in Splunk using regex extraction to validate MITRE ATT&CK techniques T1047 (WMI) and T1021.002 (SMB/Admin Shares).
- **Automated AI Triage:** The pipeline was finalized by constructing a custom Python script that parsed raw Sysmon JSON payloads and queried a local LLM (`qwen2:0.5b`) running via Ollama on an Ubuntu SOC server. The model successfully output a structured operational threat report complete with MITRE mapping, severity evaluation, and containment steps.

Ultimately, this exercise proves how behavioral endpoint telemetry can be coupled with lightweight local AI automation to drastically reduce threat triage time in modern Security Operations Centers (SOCs) without relying on external cloud APIs.

Used Gemini as a co-pilot to help work through concepts, troubleshoot setup issues, and dump/format the documentation. 🤖